

Portage du Bytestream De- coder sur Coyote

**Master of Science HES SO in
Engineering**

Profil/Orientation Computer Science

Date of publication: 04.06.2026

Produced by:

Author Abivarman Kandiah

Under the supervision of Andres Upegui, Quentin Berthet



Table of Contents

Declaration of authenticity	3
Resume (English version)	4
Résumé (version française)	4
Liste des abréviations	5
1. Introduction	6
2. Contexte technique	7
2.1. Accélération matérielle par FPGA pour le traitement de flux	7
2.2. Frameworks de développement hôte-FPGA	7
2.2.1. XRT	8
2.2.2. Coyote	8
2.3. Bytestream Decoder	9
3. Prise en main de Coyote	10
3.1. Mise en place de l'environnement	10
3.2. Tests sur U55C — difficultés rencontrées	10
3.3. Tests sur V80 — difficultés rencontrées	11
4. Portage du Bytestream Decoder sur Coyote	12
4.1. Adaptation du wrapper hardware (Design 1)	12
4.2. Adaptation du code hôte	12
4.3. Design 2 — sortie 128 bits	14
5. Analyse des performances et optimisation	15
5.1. Méthodologie de mesure (ILA)	15
5.2. Design 1 — diagnostic du goulot	15
5.3. Design 2 — gain de performance	16
5.4. Montée en fréquence à 400 MHz	17
5.5. Comparaison avec XRT	17
5.6. Tableau comparatif et bilan	17
6. Conclusion	18
Bibliography	20
A Dépôts de code	22





Declaration of authenticity

I hereby affirm that this assignment is my own written work and that I have used no other sources or aids other than those indicated. All passages that have been quoted from publications or paraphrased from these sources are indicated as such. Additionally, any assistance from artificial intelligence tools has been appropriately acknowledged and documented.

Date : 04.06.2026

Name : Abivarman Kandiah



report.typ	V1.0		
HES-SO / Domaine I&A / MSE / Rapport de Projet d'approfondissement	04-06-2026 - Abivarman Kandiah	- 3 / 22 -	

Resume (English version)

This project evaluates **Coyote**, an open-source framework developed at ETH Zürich for FPGA-to-host communication over PCIe, as an alternative to XRT, AMD/Xilinx's official tool. The evaluation is built around a concrete use case: porting the **Bytestream Decoder**, an algorithm from the ATLAS experiment at CERN whose FPGA implementation in VHDL already existed under XRT.

The framework was validated on two boards. The Alveo U55C remained unstable for reasons that could not be identified within the project's timeframe. The Versal V80, by contrast, operates reliably under Coyote once a MSI-X fix is applied in the kernel boot arguments. The V80 is not supported by XRT, which makes Coyote the only viable option on that board.

The port itself required no changes to the decoder's VHDL core, only the SystemVerilog wrapper and the host code around the Coyote API had to be rewritten. A Design 2 variant, widening the output bus from 32 to 128 bits, delivered a $\times 3.8$ speedup on transfer time. At an equivalent design, Coyote is already slightly faster than XRT thanks to the direct streaming between the host and the FPGA memory, which avoids the round-trip through the card's HBM/DDR memory.

Résumé (version française)

Ce projet évalue **Coyote**, un framework open source développé à l'ETH Zürich pour faire communiquer un FPGA avec son hôte via PCIe, comme alternative à XRT, l'outil officiel d'AMD/Xilinx. L'évaluation se fait sur un cas concret : le portage du **Bytestream Decoder**, un algorithme issu de l'expérience ATLAS au CERN dont une implémentation FPGA en VHDL existait déjà sous XRT.

Le framework a été validé sur deux cartes. L'Alveo U55C est restée instable pour des raisons qui n'ont pas pu être identifiées dans le temps imparti. La Versal V80, en revanche, fonctionne de manière stable et reproductible une fois un fix MSI-X appliqué dans les bootargs du noyau. La V80 n'étant pas supportée par XRT, Coyote est par ailleurs la seule option viable sur cette carte.

Le portage proprement dit n'a pas demandé de modifier le cœur VHDL du décodeur, seuls le wrapper SystemVerilog et le code hôte autour de l'API Coyote ont été refaits. Une variante Design 2, élargissant la sortie de 32 à 128 bits, a apporté un gain d'un facteur $\times 3.8$ sur le temps de transfert. À design équivalent, Coyote est déjà légèrement plus rapide que XRT grâce au streaming direct entre la mémoire hôte et le FPGA, sans copies par la mémoire HBM/DDR de la carte.

Liste des abréviations

AMD	Advanced Micro Devices
API	Application Programming Interface
AXI	Advanced eXtensible Interface — protocole de bus AMBA
BIOS	Basic Input/Output System
CPU	Central Processing Unit — processeur
DDR	Double Data Rate — mémoire à double débit
DMA	Direct Memory Access — accès direct à la mémoire
FEB	Front-End Board — carte d'acquisition du détecteur ATLAS
FIFO	First In, First Out — file d'attente à ordre conservé
FPGA	Field-Programmable Gate Array — circuit logique reprogrammable
GPU	Graphics Processing Unit — processeur graphique
GRUB	GRand Unified Bootloader — chargeur de démarrage Linux
HBM	High Bandwidth Memory — mémoire à large bande passante
HLS	High-Level Synthesis — synthèse matérielle à partir d'un langage de haut niveau
ILA	Integrated Logic Analyzer — analyseur logique intégré (IP Xilinx)
IOMMU	Input-Output Memory Management Unit
IP	Intellectual Property block — brique réutilisable de design FPGA
LAr	Liquid Argon — calorimètre à argon liquide d'ATLAS
LUT	Look-Up Table — table de correspondance
MSI-X	Message Signaled Interrupts eXtended — mécanisme d'interruptions PCIe
NoC	Network-on-Chip — réseau sur puce
PCI	Peripheral Component Interconnect
PCIe	PCI Express
vFPGA	virtual FPGA — instance applicative dans le shell Coyote
VHDL	VHSIC Hardware Description Language
XRT	Xilinx Runtime — framework officiel d'AMD/Xilinx

1. Introduction

Pour faire communiquer un FPGA avec un hôte via PCIe, il faut mettre en place toute une infrastructure côté FPGA : contrôleur PCIe, DMA, gestion mémoire, chargement de la logique applicative. C'est un travail long et fastidieux, qui mobilise du temps de design sans rapport direct avec l'application visée.

Des frameworks comme XRT ou Coyote viennent justement résoudre ce problème. Ils fournissent une couche d'abstraction prête à l'emploi qui gère ces aspects génériques, et laissent l'utilisateur se concentrer directement sur la logique applicative. C'est particulièrement intéressant pour du prototypage rapide, où l'on veut pouvoir itérer ou changer de logique sans toucher à l'infrastructure.

Dans l'écosystème AMD/Xilinx, ce rôle est historiquement tenu par **XRT**, l'outil officiel d'AMD. Mais un nouveau framework, **Coyote**, est apparu plus récemment et propose des possibilités qui pourraient à terme rendre XRT obsolète.

Pour évaluer concrètement Coyote, il faut un cas d'usage représentatif. Le **Bytestream Decoder**, un algorithme utilisé dans la chaîne de lecture du calorimètre LAr de l'expérience ATLAS au CERN, remplit ce critère. Mr. Upegui en a implémenté une version FPGA en VHDL, déjà validée sous XRT sur cartes Alveo. Ce design existant offre donc un point de comparaison direct entre les deux frameworks.

Trois objectifs ont été fixés pour ce projet :

1. Prendre en main Coyote et valider son fonctionnement sur les deux cartes à disposition : l'Alveo U55C et la Versal V80.
2. Porter le Bytestream Decoder de XRT vers Coyote, en évaluant l'effort nécessaire et les écarts de performance.
3. Identifier les forces, les limites et les perspectives de Coyote sur ce type d'algorithme.

Le rapport est organisé comme suit. Le chapitre *Contexte technique* pose les bases : FPGA, frameworks XRT et Coyote, et présentation du Bytestream Decoder. Le chapitre *Prise en main et évaluation de Coyote* documente la mise en route du framework sur les deux cartes et les difficultés rencontrées. Le chapitre *Portage du Bytestream Decoder sur Coyote* détaille l'adaptation du wrapper hardware, la réécriture du code hôte, et l'introduction d'une variante Design 2 à sortie 128 bits. Le chapitre *Analyse des performances et optimisation* présente la méthodologie de mesure via ILA, le diagnostic du goulot, la montée en fréquence à 400 MHz et la comparaison directe avec l'implémentation XRT d'origine. Le rapport se termine par une synthèse et l'évocation des prolongements possibles.

2. Contexte technique

2.1. Accélération matérielle par FPGA pour le traitement de flux

Les FPGAs sont des matériels utiles à des applications nécessitant des traitements de données à très faible latence, avec des exigences de débit élevées, et une flexibilité d'architecture

Contrairement aux CPUs et aux GPUs, dont l'architecture est figée et le parallélisme borné par le nombre de cœurs ou de threads, un FPGA permet de décrire une architecture sur mesure, dans laquelle chaque étage de traitement s'exécute en parallèle des autres. Cette propriété rend les FPGAs particulièrement adaptés au traitement de flux de données en continu.

Deux cartes ont été utilisées dans le cadre de ce projet. La première, l'Alveo U55C, est une carte d'accélération basée sur un FPGA UltraScale+; La seconde, la V80, est plus récente : elle repose sur l'architecture Versal (NoC matériel intégré, AI Engines), expose une interface PCIe Gen5 x8 et embarque également de la HBM [1].

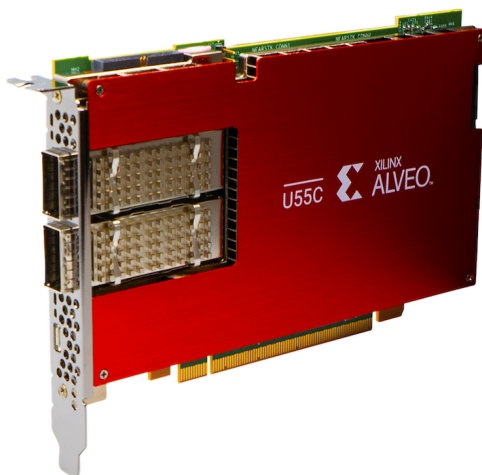


Figure 1: Xilinx Alveo U55C, une carte d'accélération basée sur un FPGA UltraScale+.

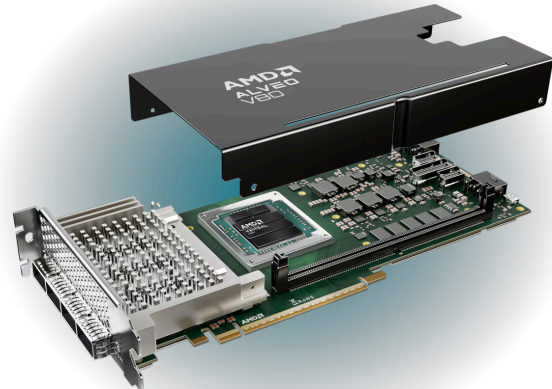


Figure 2: Xilinx Versal V80, une carte d'accélération basée sur un FPGA Versal.

2.2. Frameworks de développement hôte-FPGA

Pour faciliter le développement et le déploiement d'applications sur FPGA, ils existent des frameworks nous fournissant un shell qui est une sorte de système d'exploitation pour FPGA, qui gère les aspects génériques de la communication avec l'hôte (PCIe, DMA, gestion mémoire, etc.) laissant à l'utilisateur la liberté de se concentrer sur la logique applicative. Le shell va généralement permettre à l'utilisateur de charger dynamiquement une application logique sur le FPGA, sans avoir à reprogrammer le FPGA en entier.

Ces frameworks fournissent également une API pour l'hôte, permettant de faciliter les interactions avec le FPGA tel que le chargement de la logique applicative et le transfert de données.

2.2.1. XRT

XRT est le framework officiel d'AMD/Xilinx qui va cibler principalement leurs cartes FPGA [2]. La programmation de la couche logique applicative repose sur des kernels généralement décrits en HLS. La transition des données entre l'hôte et le FPGA suit un schéma comme celui-ci :

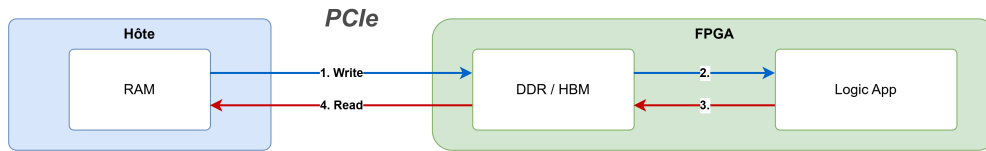


Figure 3: Schéma du chemin de données entre l'hôte et le FPGA sous XRT.

XRT bénéficie d'un écosystème mature, intégré à la toolchain Vitis, mais malheureusement ne supporte pas les cartes plus récentes comme la V80.

2.2.2. Coyote

Coyote est un framework open source développé à l'ETH Zürich [3]. La logique applicative y prend la forme d'un ou plusieurs **vFPGAs** (*virtual FPGAs*), qui dialoguent avec le shell via des interfaces AXI-Stream. Plusieurs vFPGAs peuvent coexister dans le même shell, ce qui permet d'isoler plusieurs applications sur le même FPGA.

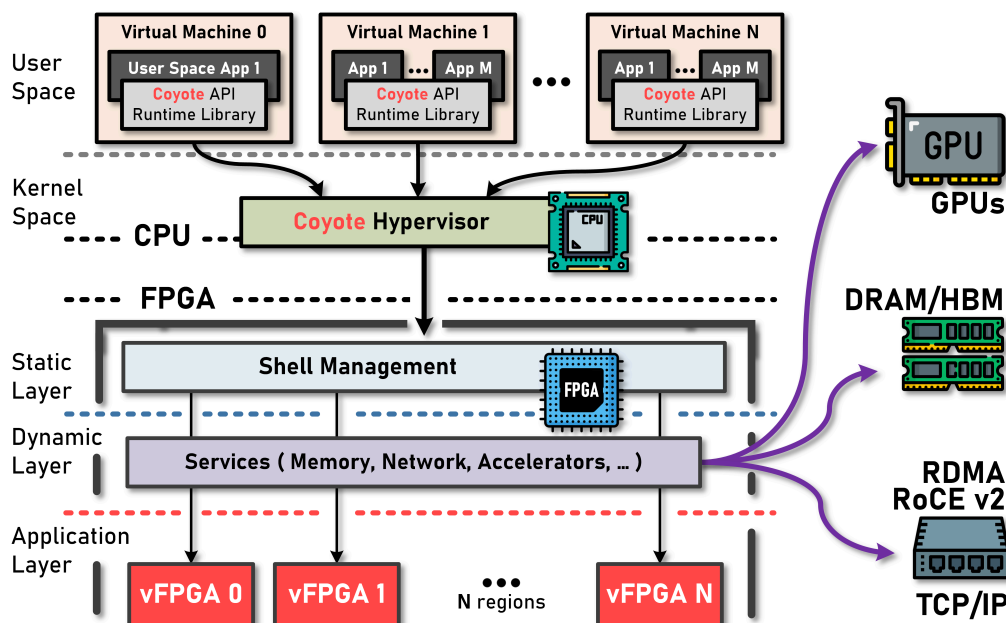


Figure 4: Architecture d'un shell Coyote avec un vFPGA. Source: <https://github.com/fpgasystems/Coyote/tree/master>

Le top-level du vFPGA s'écrit en SystemVerilog, mais la logique applicative en dessous peut être décrite en VHDL, Verilog, SystemVerilog, HLS, ou encore SpinalHDL.

Coyote propose également plusieurs modes de transfert de données entre l'hôte et le FPGA, comme illustré sur le schéma suivant :

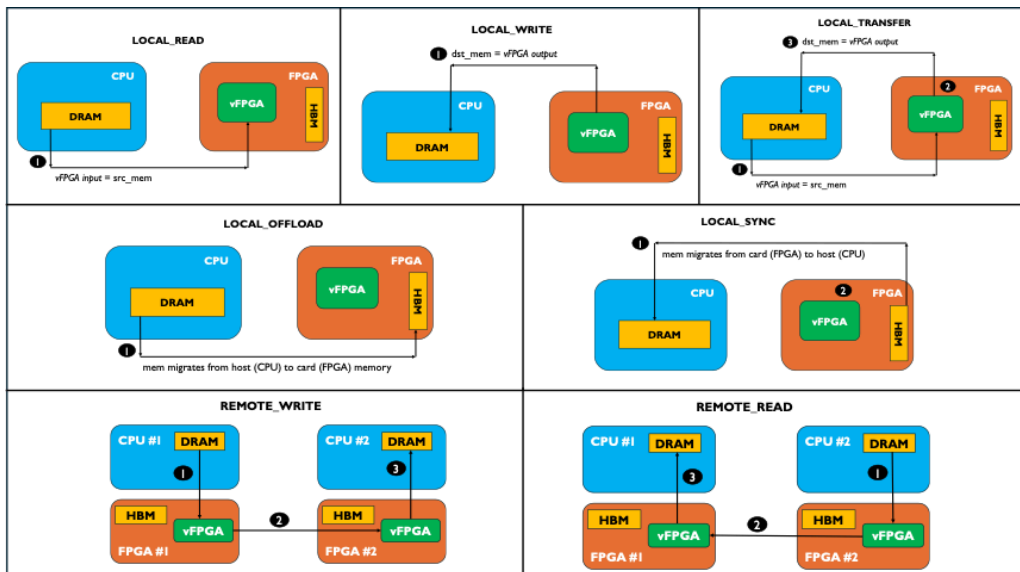


Figure 5: Schéma des différents modes de transfert de données entre l'hôte et le FPGA proposés par Coyote. Source: <https://github.com/fpgasystems/Coyote/tree/master>

Le mode de transfert qui va nous intéresser en particulier c'est le mode "Local Read/Write/Transfer" qui va permettre de streamer les données directement entre la mémoire de l'hôte et le vFPGA, sans passer par une mémoire intermédiaire sur la carte. Enfin, Coyote supporte depuis récemment la V80, ce qui en fait à ce jour l'une des rares options viables pour exploiter cette carte.

2.3. Bytestream Decoder

L'algorithme retenu pour tester Coyote sur un cas concret est le **Bytestream Decoder**, développé pour l'expérience ATLAS au CERN [4]. Il sert dans la chaîne de lecture du calorimètre à argon liquide (LAR), en tête du pipeline d'accélération du **Topo-automaton clustering**.

Le flux d'entrée contient, sous forme brute, les données mesurées pour chaque cellule du calorimètre : gain, énergie, temps et qualité. Le décodeur va lire ce flux et reconstruire à la volée une structure exploitable côté logiciel, appelée **CaloCell container**.

Cette implémentation est intéressante car le décodeur existe déjà sous la forme d'un design VHDL validé sous XRT sur des cartes Alveo [5], ce qui va nous donner un point de comparaison direct avec Coyote.

Le pipeline prend en entrée deux flux : le ByteStream brut (700 KB) et des données de correction (7 KB).

En interne, le pipeline travaille sur un bus de 32 bits. Le module expose donc deux flux AXI-Stream 32 bits en entrée, et produit en sortie quatre mots de 32 bits par cellule : Gain+ID, Energy, Time, Quality. Cette largeur de 32 bits, héritée de l'implémentation XRT d'origine, va jouer un rôle important dans l'analyse de performance présentée plus loin.

3. Prise en main de Coyote

3.1. Mise en place de l'environnement

Le flot de développement avec Coyote se base sur CMake et Vivado 2024.2 (pour la V80). Le design hardware se build via CMake en précisant la carte cible avec `-DFDEV_NAME=u55c` ou `-DFDEV_NAME=v80`.

Le driver noyau se compile à part. Pour la V80, il faut préciser `TARGET_PLATFORM=versal` au `make`, sinon le driver généré ne correspond pas à la carte.

Un cycle de déploiement complet ressemble donc à ça :

1. build du bitstream
2. programmation du FPGA via Vivado
3. remove et rescan du bus PCIe (sinon le device n'est pas redéetecté après reprogrammation)
4. chargement du driver Coyote
5. exécution du programme hôte

Pour valider la chaîne, j'ai commencé par l'exemple `01_hello_world` fourni avec Coyote, qui réalise un simple transfert hôte ↔ FPGA. C'est sur cet exemple que la majorité des problèmes de mise en route ont été rencontrés.

3.2. Tests sur U55C — difficultés rencontrées

Deux problèmes principaux sont apparus lors de la mise en route sur l'Alveo U55C.

Device PCIe non détecté après reprogrammation du FPGA. Quand on reprogramme le FPGA, le device PCIe n'est plus directement utilisable côté hôte, ce qui fait planter le programme de test avec une erreur du type `cThread instance could not be obtained`. Au départ, je ne faisais pas la bonne manipulation pour réinitialiser le bus PCIe. Il faut en réalité forcer un **remove** du device, puis un **rescan** du bus, avant de recharger le driver :

```
sudo sh -c "echo 1 > /sys/bus/pci/devices/0000:21:00.0/remove"
sudo sh -c "echo 1 > /sys/bus/pci/rescan"
```

Programme hôte bloquant pendant le transfert. Le design est programmé, le driver se charge sans erreur, mais le programme hôte reste bloqué pendant le transfert. Initialement, on pensait que le problème venait de l'IOMMU qui n'était pas en mode passthrough. Ajouter `amd_iommu=on iommu=pt` dans les bootargs GRUB semblait fixer le souci au début, mais au final non : le comportement est resté instable.

Dans certains cas, même avec le bitstream et le driver chargés correctement, le programme hôte se bloque toujours sur le transfert. Une analyse avec l'ILA montre que le signal `tvalid` de l'interface réceptrice ne passe jamais à 1, donc le transfert ne démarre pas du tout côté FPGA. La cause exacte n'a pas pu être identifiée dans le temps imparti.

Au final, la U55C n'est pas fiable pour faire des mesures reproductibles, ce qui nous a poussé à utiliser la V80 comme plateforme principale.

3.3. Tests sur V80 — difficultés rencontrées

La V80 venait d'être officiellement supportée par Coyote au moment du projet, donc valider ce support faisait partie des objectifs.

Le problème principal rencontré était au chargement du driver, avec une erreur liée au MSI-X. La carte expose un grand nombre de vecteurs d'interruption, et le BIOS ne réserve pas assez de mémoire PCI pour les allouer. Le fix consiste à ajouter `pci=realloc=on` dans les bootargs GRUB pour forcer le kernel à réallouer les ressources PCI au boot.

Une fois ce problème réglé, la V80 fonctionne de manière stable et reproductible. À noter cependant qu'à chaque reprogrammation de la V80 via Vivado, il peut être nécessaire de redémarrer la machine hôte pour forcer une réallocation propre du PCI au boot, sinon on peut rencontrer à nouveau cette erreur.

C'est cette carte qui a servi de plateforme pour toutes les mesures de performance présentées plus loin.

4. Portage du Bytestream Decoder sur Coyote

Le portage du Bytestream Decoder de XRT vers Coyote consiste à refaire le wrapper au format attendu par Coyote, et réécrire le code hôte autour de l'API Coyote. Le cœur VHDL du décodeur, lui, reste identique à la version XRT d'origine.

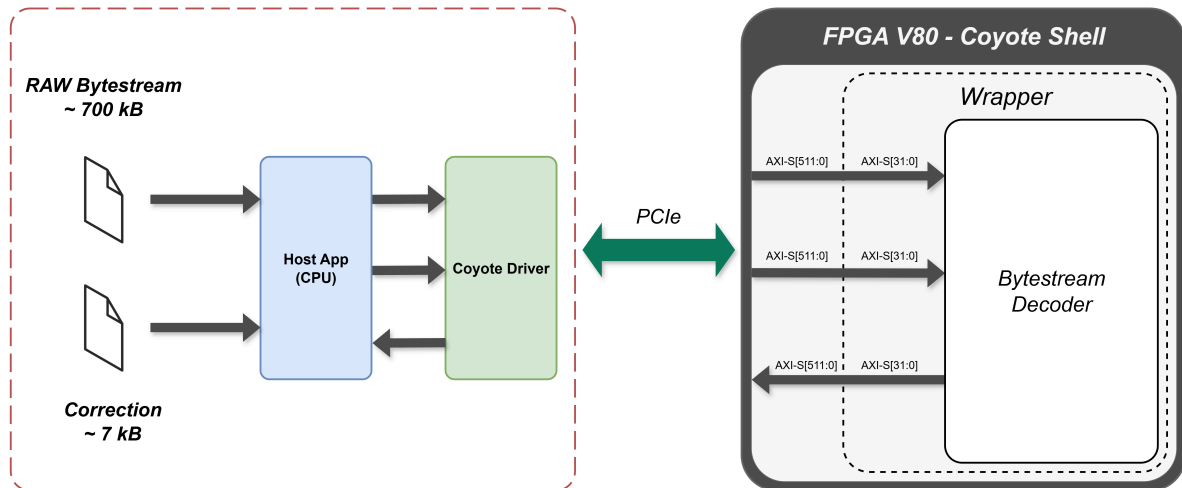


Figure 6: Vue d'ensemble du portage : chargement des fichiers et packing côté hôte, transfert via Coyote, décodage dans le vFPGA, puis retour vers l'hôte.

4.1. Adaptation du wrapper hardware (Design 1)

Le shell Coyote expose un bus AXI-Stream de 512 bits côté vFPGA, alors que le Bytestream Decoder travaille en interne sur des mots de 32 bits. Le rôle du wrapper, écrit en SystemVerilog dans `vfpga_top.svh`, est donc d'instancier le module VHDL et de connecter ses ports 32 bits aux bits de poids faible de chaque interface AXI-Stream.

Concrètement :

- En entrée, seuls les bits [31:0] des flux `axis_host_recv[0]` (données brutes) et `axis_host_recv[1]` (corrections initiales) sont connectés au décodeur.
- En sortie, les bits [31:0] du flux `axis_host_send[0]` portent un mot de sortie par cycle.
- Les 480 bits restants de chaque bus ne sont pas utilisés. Le bus est donc sous-utilisé d'un facteur 16, ce qui est volontaire pour ce premier design.

Le wrapper instancie aussi une ILA décrit via le script `init_ip.tcl`, avec des sondes sur les signaux axi-stream des interfaces d'entrée et de sortie. Cela va permettre de mesurer les duty cycles côté FPGA et d'identifier les goulots dans l'analyse de performance présentée plus loin.

4.2. Adaptation du code hôte

Le code hôte a été entièrement réécrit autour de l'API Coyote, à la place du modèle `xclbin` + buffers de XRT.

Le programme commence par charger les deux fichiers d'entrée :

- `Uncompressed_data_LAR_only.raw` : 174 736 mots de 32 bits (données brutes du bytestream)

- `initial_correction_values.bin` : 1 833 mots de 32 bits (valeurs de correction)

Coyote travaillant sur des blocs de 512 bits, chaque mot 32 bits est placé dans les bits de poids faible d'un bloc de 64 octets, les 60 octets restants étant remplis de zéros.

Les données sont ensuite envoyées au FPGA via deux flux distincts :

- Stream 0 : les données brutes du bytestream
- Stream 1 : les valeurs de correction, envoyées une seule fois en début d'exécution

Le transfert principal est déclenché par deux appels à `coyote_thread.invoke()` : un `LOCAL_READ` pour pousser les données brutes vers le FPGA, et un `LOCAL_WRITE` pour récupérer la sortie. Le programme attend ensuite la complétion des deux opérations.

Côté sortie, chaque CaloCell occupe quatre blocs de 512 bits successifs, à raison d'un mot 32 bits par bloc : Gain+ID, Energy, Time, puis Quality. L'unpacking parcourt donc le buffer de sortie quatre blocs à la fois pour reconstruire chaque cellule.

4.3. Design 2 — sortie 128 bits

La sortie du décodeur, une cellule **CaloCell**, est constituée de 4 mots de 32 bits. Dans le Design 1, ces 4 mots sont émis un par un par l' `Output_Merger_32`, ce qui demande 4 blocs de 512 bits sur le bus AXI-Stream pour transférer en réalité seulement 128 bits utiles.

L'idée du Design 2 est de les regrouper en un seul mot de 128 bits, et de les envoyer directement dans un seul bloc de 512 bits. On passe ainsi de 4 blocs à 1 bloc par cellule, soit un facteur $\times 4$ sur la bande passante utile en sortie.

À noter que la variante 128 bits du merger (`Output_Merger_128`) était déjà présente dans le code source VHDL, mais c'est la variante 32 bits (`Output_Merger_32`) qui était utilisée pour le portage sous XRT. Les deux versions cohabitent donc dans `Byte_Stream_Decoder_parallel.vhd`, et il a suffi de swapper l'instanciation pour activer la variante 128 bits.

Côté wrapper, le changement se résume à connecter les bits `[127:0]` du flux de sortie au lieu des bits `[31:0]`. Les 384 bits restants du bloc ne sont toujours pas utilisés, mais le ratio passe de 1/16 à 4/16.

Côté hôte, l'unpacking est ajusté pour lire une cellule complète par bloc, en extrayant les 4 champs (Gain+ID, Energy, Time, Quality) dans les 16 octets de poids faible.

Le reste de la chaîne reste strictement identique au Design 1.

5. Analyse des performances et optimisation

5.1. Méthodologie de mesure (ILA)

Pour analyser le comportement du pipeline côté FPGA, une IP ILA (*Integrated Logic Analyzer*) a été instanciée dans le wrapper du vFPGA. Elle sonde les signaux `tvalid`, `tready`, `tlast` et `tdata` des interfaces AXI-Stream d'entrée (`axis_host_recv[0]`) et de sortie (`axis_host_send[0]`).

L'ILA est configurée pour déclencher sur le `tvalid` de l'interface de réception et capturer 512 échantillons post-trigger, ce qui couvre une fenêtre représentative du transfert.

Deux ratios sont particulièrement intéressants à observer :

- **tready entrée** : indique la proportion de cycles pendant lesquels le FPGA est prêt à consommer une donnée. S'il est faible, c'est que le pipeline interne ne peut pas suivre, et le goulot est en aval.
- **tvalid sortie** : indique la proportion de cycles pendant lesquels le FPGA a une donnée à émettre. S'il est élevé, c'est que le pipeline produit sans blocage.

Le `tvalid` côté entrée et le `tready` côté sortie sont eux quasiment toujours à 1 dans nos mesures : l'hôte Coyote a toujours des données prêtes à envoyer, et toujours assez de place pour absorber la sortie.

5.2. Design 1 — diagnostic du goulot

Le Design 1 a été synthétisé à 250 MHz avec la sortie 32 bits héritée du portage XRT. Le temps total mesuré côté hôte pour le transfert principal est de 3.17 ms.

Pour comprendre où se situe le goulot, on regarde les 4 signaux de handshake AXI-Stream côté FPGA (Figure 7) :

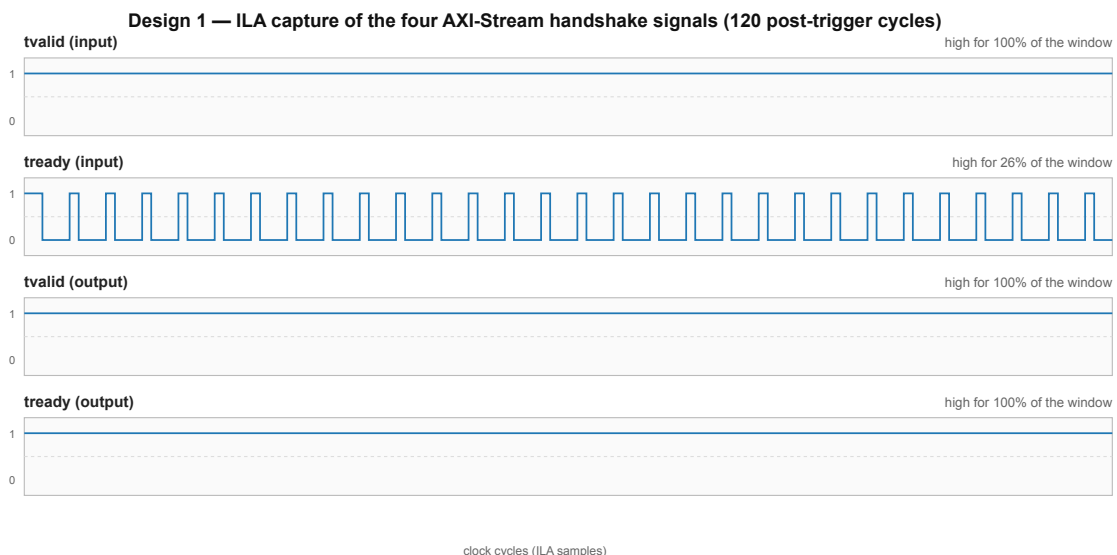


Figure 7: Capture ILA des quatre signaux de handshake AXI-Stream du Design 1, à 250 MHz, sur 120 cycles post-trigger. Le `tready` de l'entrée est le seul signal qui pulse ; les trois autres restent collés à 1 sur toute la fenêtre.

Sur la fenêtre complète de 512 échantillons capturés, le `tvalid` de l'entrée et les `tvalid` / `tready` de la sortie sont à 100 %. Seul le `tready` de l'entrée descend, à 20.9 %.

L'interprétation est directe. L'hôte a toujours des données à envoyer (`tvalid` entrée à 1), le FPGA a toujours une donnée à émettre (`tvalid` sortie à 1), et l'hôte est toujours prêt à recevoir (`tready` sortie à 1). Le seul facteur limitant est donc le FPGA qui ne peut absorber l'entrée que pendant environ un cinquième du temps. Le goulot est clairement en aval, plus loin dans le pipeline.

Cela correspond bien à ce qu'on attendait : la sortie émet un mot 32 bits par cycle, alors qu'une cellule complète en compte quatre. Le merger met donc 4 cycles pour évacuer une cellule, pendant lesquels les étages amont sont gelés.

5.3. Design 2 — gain de performance

Avec l'`Output_Merger_128`, à la même fréquence de 250 MHz, le temps de transfert mesuré côté hôte chute de 3.17 ms à 0.83 ms, soit un gain d'un facteur $\times 3.8$.

Côté ILA, les ratios deviennent : `tready` entrée à 88.7 % et `tvalid` sortie à 99.4 %. Le pipeline consomme maintenant l'entrée la quasi-totalité du temps, ce qui valide le diagnostic posé sur le Design 1.

La figure Figure 8 illustre visuellement cette différence de comportement entre les deux designs sur le signal `tready` de l'entrée.

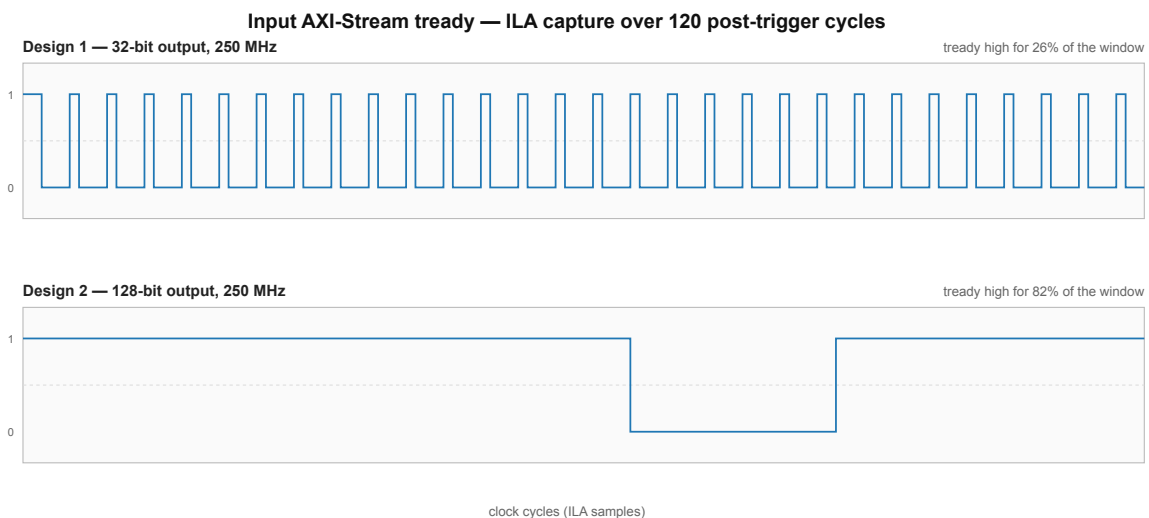


Figure 8: Capture ILA du signal `tready` de l'AXI-Stream d'entrée sur le Design 1 (haut) et le Design 2 (bas), à 250 MHz. Sur le Design 1, le signal pulse périodiquement (un cycle haut sur quatre) ; sur le Design 2, il reste à 1 la quasi-totalité du temps. La fenêtre affichée couvre 120 cycles post-trigger ; les ratios cités dans le texte (20.9 % / 88.7 %) sont calculés sur la fenêtre complète de 512 échantillons.

5.4. Montée en fréquence à 400 MHz

Une fois le goulot de sortie levé, la question se pose de savoir si monter la fréquence du pipeline permet de gagner encore en débit. Le Design 2 a donc été resynthétisé à 400 MHz, en paramétrant la fréquence cible côté CMake.

Le temps de transfert tombe à 0.82 ms, soit un gain marginal d'environ 1 % par rapport au Design 2 à 250 MHz.

Côté ILA, les ratios sont eux aussi très proches du cas 250 MHz : `tready` entrée à 90.0 % et `tvalid` sortie à 99.4 %. Cela signifie que le pipeline FPGA n'est plus le facteur limitant. Le goulot restant est ailleurs : soit dans le transfert PCIe lui-même, soit dans la sous-utilisation persistante du bus d'entrée (toujours 1 mot 32 bits par bloc de 512 bits, soit 1/16). Monter la fréquence ne change rien à ces deux facteurs.

5.5. Comparaison avec XRT

Pour situer ces résultats par rapport au point de départ, on peut les comparer à l'implémentation XRT d'origine du Bytestream Decoder, qui tournait sur une carte VCK5000 à 350 MHz avec une sortie 32 bits.

Sur cette implémentation, le programme hôte rapporte les temps suivants :

- écriture des données vers la DDR de la carte : 0.26 ms
- écriture des paramètres de correction : 0.28 ms
- temps total du kernel : 2.94 ms
- lecture des résultats depuis la DDR : 0.59 ms
- temps total bout-en-bout : 4.12 ms

Deux différences importantes sont à garder à l'esprit pour interpréter cette comparaison.

D'une part, les cartes ne sont pas les mêmes. Une partie des écarts de performance peut donc venir du matériel et non du framework.

D'autre part, le modèle de transfert n'est pas le même. Sous XRT, les données doivent transiter par la mémoire DDR/HBM de la carte avant d'être consommées par le kernel, et la sortie suit le chemin inverse. Coyote permet à l'inverse de streamer directement la mémoire hôte vers le vFPGA, ce qui évite les deux étapes d'écriture et de lecture en mémoire embarquée. C'est précisément le mode "Local Read/Write" mentionné dans le contexte technique.

À design équivalent (sortie 32 bits, fréquence comparable), le temps total bout-en-bout est dans le même ordre de grandeur : 4.12 ms sous XRT contre 3.17 ms pour le Design 1 sous Coyote. La différence vient principalement de l'élimination des copies DDR. Une fois le Design 2 introduit, l'écart se creuse nettement : 0.83 ms sous Coyote, soit environ $\times 5$ par rapport à XRT.

5.6. Tableau comparatif et bilan

Le tableau Table 1 récapitule les configurations testées et leur point de comparaison XRT.

Framework	Carte	Fréquence	Sortie	Temps total	tready entrée
XRT	VCK5000	350 MHz	32 bits	4.12 ms	—
Coyote (Design 1)	V80	250 MHz	32 bits	3.17 ms	20.9 %
Coyote (Design 2)	V80	250 MHz	128 bits	0.83 ms	88.7 %
Coyote (Design 2)	V80	400 MHz	128 bits	0.82 ms	90.0 %

Table 1: Comparaison des performances de l'implémentation XRT d'origine et des configurations Coyote testées sur V80.

Trois conclusions ressortent de cette analyse.

D'abord, à design équivalent (sortie 32 bits), Coyote est déjà légèrement plus rapide que XRT bout-en-bout, principalement grâce au streaming direct mémoire hôte ↔ vFPGA qui évite les copies DDR.

Ensuite, l'optimisation décisive vient de l'alignement entre la largeur de sortie du pipeline et celle du bus Coyote, pas de la fréquence. Passer de l' `Output_Merger_32` à l' `Output_Merger_128` apporte un facteur ×3.8 sur le temps de transfert, alors que doubler la fréquence n'apporte qu'environ 1 % supplémentaire.

La prochaine optimisation naturelle serait d'élargir aussi le bus d'entrée, qui n'utilise actuellement qu'un mot 32 bits sur 512 disponibles par bloc.

6. Conclusion

L'objectif principal de ce projet était d'évaluer Coyote comme alternative à XRT, sur un cas d'usage concret, et de valider son support sur la V80 récemment ajouté. Sur les trois axes annoncés en introduction, le bilan est globalement positif.

D'abord, la **prise en main de Coyote** a été menée sur les deux cartes à disposition. La V80 fonctionne de manière stable et reproductible une fois le fix `pci=realloc=on` appliqué dans les bootargs GRUB, ce qui constitue une première validation pratique du support récent de cette carte par le framework. La U55C, en revanche, est restée instable : le bitstream se programme, le driver se charge, mais le `tvalid` de l'AXI-Stream d'entrée ne démarre jamais dans certains cas, sans cause identifiée dans le temps imparti.

Ensuite, le **portage du Bytestream Decoder** a abouti sans toucher au cœur VHDL du décodeur. L'essentiel du travail d'adaptation tient dans le wrapper SystemVerilog et dans la réécriture du code hôte autour de l'API Coyote. Une variante Design 2 a aussi été produite en élargissant la sortie de 32 à 128 bits, via le merger déjà présent dans le code VHDL d'origine. Cette modification, minime côté hardware, a apporté un facteur ×3.8 sur le temps de transfert.

Enfin, la **comparaison avec XRT** montre que Coyote est viable à design équivalent, et bénéficie en plus du streaming direct mémoire hôte ↔ vFPGA, qui évite les copies DDR imposées par XRT. Sur la V80 en particulier, c'est aujourd'hui l'une des seules options exploitables, vu que XRT ne supporte pas cette carte.

Plusieurs **difficultés** sont à signaler. Le problème MSI-X au chargement du driver V80 a demandé un fix dans les bootargs GRUB, et la reprogrammation de la carte peut encore exiger un redémarrage de

la machine hôte pour que les ressources PCI soient réallouées proprement. Côté U55C, l'instabilité au démarrage des transferts reste sans explication satisfaisante : malgré plusieurs hypothèses testées (driver, IOMMU, rescan PCIe), le `tvalid` d'entrée ne démarre toujours pas dans certains cas, et la cause n'a pas pu être isolée dans le temps imparti.

Plusieurs **pistes** d'amélioration ressortent enfin des mesures. La plus directe serait d'élargir aussi le bus d'entrée : actuellement un seul mot de 32 bits sur 512 est utilisé par bloc, soit un ratio 1/16. Packer 16 mots par bloc devrait théoriquement multiplier d'autant le débit d'ingestion, à condition que le pipeline interne du décodeur puisse suivre. Au-delà, traiter plusieurs événements en parallèle dans un même shell Coyote, en exploitant le support multi-vFPGA, constituerait un prolongement naturel.

Une piste plus ambitieuse, et plus en phase avec l'esprit de Coyote, serait d'exploiter le scénario **smartNIC**. Actuellement, le Bytestream Decoder et l'ensemble du pipeline qui le suit tournent sur GPU, et les données détecteur transitent par une carte réseau classique avant d'être copiées vers le GPU. Un FPGA Coyote installé en lieu et place de la carte réseau pourrait ingérer directement le flux du détecteur, exécuter le Bytestream Decoder à la volée, et transférer ensuite le `CaloCell container` directement dans la mémoire du GPU, sans repasser par le CPU hôte. Ce type de déploiement correspond exactement aux cas d'usage dataplane visés par Coyote.

Bibliography

- [1] AMD, “AMD Alveo V80 Compute Accelerator Card – Product Brief.” 2024.
- [2] AMD/Xilinx, “XRT: Xilinx Runtime Library Documentation.” 2024.
- [3] D. Korolija, T. Roscoe, and G. Alonso, “Do OS Abstractions Make Sense on FPGAs?,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, USENIX Association, 2020. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/roscoe>
- [4] ATLAS LAr Collaboration, “The ATLAS Liquid Argon Calorimeter: Construction, Integration, Commissioning and Performance,” *Journal of Instrumentation*, vol. 5, p. P9003, 2010.
- [5] A. Upegui Posada, “Bytestream Decoder – design VHDL pour ATLAS LAr sur Alveo U55C.” 2024.
- [6] AMD/Xilinx, “Alveo U55C High Performance Compute Card – Product Brief.” 2022.
- [7] D. Firestone and others, “Azure Accelerated Networking: SmartNICs in the Public Cloud,” in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX Association, 2018.



Appendices

A Dépôts de code	22
------------------------	----



report.typ	V1.0		
HES-SO / Domaine I&A / MSE / Rapport de Projet d'approfondissement	04-06-2026 - Abivarman Kandiah	- 21 / 22 -	



A Dépôts de code

Le code complet du portage sous Coyote est disponible sur GitHub :

https://github.com/Djokzer/coyote_pa

Le design XRT d'origine du Bytestream Decoder, développé par Andres Upegui, est disponible sur le GitLab du CERN :

https://gitlab.cern.ch/aupegui/ATLAS_ByteStreamDecoding_FPGA

